// THE COROUTINE COURT

I cancelled it. It kept running.

A coroutine I cancelled ran
for 8 more seconds.
The culprit was one line
I wrote myself.

// THE MENTAL MODEL

Cancel is a message, not a bullet

Cancelling a coroutine throws CancellationException up your suspend calls.
That exception IS the message: we are done, unwind.

// THE TRAP

The line that ate the message

```
try { api.search(query) }
catch (e: Exception) { ... }
```

CancellationException is an Exception. This eats it.

// FIX 01

Catch what you expect

Not Exception.
Catch IOException,
HttpException, the real thing.
Cancellation sails past
and does its job.

// FIX 02

Rethrow cancellation first

```
catch (e:
CancellationException)
{ throw e }
```

then your broad catch.

Let the message through.

// THE FOOTGUN

runCatching does it too

runCatching { } also catches CancellationException.
Handy, and a trap.
For must-run cleanup use withContext(NonCancellable).

// THE TAKEAWAY

Don't shoot the messenger

Cancellation is a conversation, not a kill switch. Swallow the message and the work never stops.